CODEX

FCT 2026

Plataforma de Documentación Técnica HTML5 & CSS3

Guía de Instalación y Scaffolding Inicial

Angular v21 + NestJS + MongoDB

Versión 1.0 | Mayo 2026

1. Introducción y Objetivo

Este documento es la guía de referencia para poner en marcha el proyecto Codex desde cero. Cubre la instalación de todas las herramientas necesarias, la generación del scaffolding inicial de Angular y NestJS, la estructura de carpetas siguiendo los principios del proyecto y los comandos para conectar el repositorio Git ya creado.

Requisitos del Proyecto (según PPT)

Frontend: Angular v21 con Signals y Standalone Components | Backend: NestJS con arquitectura modular | Base de Datos: MongoDB | Estilos: Sass/SCSS con Atomic Design y BEM | Auth: Google OAuth 2.0 + JWT

2. Instalación de Prerrequisitos

Antes de crear el proyecto necesitas instalar las siguientes herramientas en tu máquina. Sigue los pasos en orden.

2.1 Node.js (Motor de JavaScript)

Node.js es el entorno de ejecución que necesitan tanto Angular como NestJS para funcionar.

```
# 1. Descarga Node.js LTS desde la web oficial:
#    https://nodejs.org  ->  descarga la version LTS (recomendada)

# 2. Tras instalar, verifica en tu terminal:
node --version    # Debe mostrar v20.x.x o superior
npm --version     # Debe mostrar 10.x.x o superior
```

Consejo

Descarga siempre la versión LTS (Long Term Support). Es la versión estable recomendada para proyectos en producción.

2.2 Angular CLI

La interfaz de línea de comandos de Angular es la herramienta que usarás para generar componentes, servicios, guardias y más.

```
npm install -g @angular/cli@21

# Verifica la instalacion:
ng version
```

2.3 NestJS CLI

La CLI de NestJS permite generar rápidamente módulos, controladores, servicios y otros elementos de la arquitectura backend.

```
npm install -g @nestjs/cli

# Verifica la instalacion:
nest --version
```

2.4 MongoDB

Tienes dos opciones para usar MongoDB. Se recomienda la opción cloud (MongoDB Atlas) para empezar rápido:

A	**MongoDB Atlas (Cloud - Recomendado)**	Crea una cuenta gratuita en https://cloud.mongodb.com. Crea un cluster M0 (gratuito). Copia la cadena de conexión (URI).
B	**MongoDB Local**	Descarga desde https://www.mongodb.com/try/download/community. Instala y ejecuta: mongod --dbpath /data/db

2.5 Resumen de herramientas

Tecnología	Versión	Comando de instalación
Node.js	v20+ LTS	https://nodejs.org (instalador gráfico)
Angular CLI	v21	npm install -g @angular/cli@21
NestJS CLI	latest	npm install -g @nestjs/cli
MongoDB	v7+	https://cloud.mongodb.com (Atlas) o local
Git	latest	https://git-scm.com (instalador gráfico)

3. Creación del Scaffolding del Proyecto

Con todas las herramientas instaladas, ya podemos generar la estructura base del proyecto. El proyecto se divide en dos aplicaciones independientes: el frontend (Angular) y el backend (NestJS).

3.1 Estructura General de Carpetas

Crearemos una carpeta raíz que contendrá ambas aplicaciones:

```
codex-fct-2026/
├── frontend/          <- Aplicacion Angular
└── backend/           <- Aplicacion NestJS
```

3.2 Crear el Frontend (Angular v21)

Ejecuta los siguientes comandos en tu terminal:

```
# 1. Crea la carpeta raiz del proyecto
mkdir codex-fct-2026
cd codex-fct-2026

# 2. Genera el proyecto Angular
ng new frontend --routing --style=scss --standalone

# Cuando pregunte:
#   ? Which stylesheet format?  ->  SCSS
#   ? Enable SSR?               ->  No (por ahora)
```

Flags importantes explicados

--routing: Genera el módulo de rutas automáticamente. | --style=scss: Configura SCSS como preprocesador (requerido por el PPT). | --standalone: Genera componentes standalone (requerido por el PPT).

3.3 Crear el Backend (NestJS)

```
# Desde la carpeta raiz codex-fct-2026/
nest new backend

# Cuando pregunte el gestor de paquetes:
#   -> Selecciona npm
```

3.4 Instalar dependencias clave del Frontend

```
cd frontend

# Resaltado de sintaxis (requerido en RF-05)
npm install prismjs
npm install @types/prismjs --save-dev

# Renderizado de Markdown (requerido en RF-04)
npm install marked

# Vuelve a la raiz
cd ..
```

3.5 Instalar dependencias clave del Backend

```
cd backend

# MongoDB con Mongoose (persistencia)
npm install @nestjs/mongoose mongoose

# Autenticacion JWT (requerido en RF-06 y RNF-04)
npm install @nestjs/jwt passport passport-jwt @nestjs/passport
npm install @types/passport-jwt --save-dev

# Google OAuth 2.0 (requerido en RF-01)
npm install passport-google-oauth20
npm install @types/passport-google-oauth20 --save-dev

# Validacion de DTOs (requerido por el PPT)
npm install class-validator class-transformer

# Variables de entorno
npm install @nestjs/config

cd ..
```

4. Estructura de Carpetas Detallada

Tras el scaffolding y siguiendo los principios de Atomic Design (RNF-01) y la arquitectura modular de NestJS, la estructura final debe quedar así:

4.1 Frontend (Angular) — Atomic Design

```
frontend/src/
├── app/
│   ├── core/                   <- Servicios singleton,
interceptores, guards
│   │   ├── guards/
│   │   │   └── auth.guard.ts    <- Functional Guard (requerido por
PPT)
│   │   ├── interceptors/
│   │   └── services/
│   │       ├── auth.service.ts
│   │       └── lesson.service.ts
│   ├── shared/                 <- Componentes reutilizables
│   │   ├── atoms/              <- Boton, Badge, Input...
│   │   ├── molecules/          <- Buscador, Alerta, MenuItem...
│   │   └── organisms/          <- Navbar, Sidebar, LessonGrid...
│   ├── features/               <- Paginas de la app
│   │   ├── auth/
│   │   │   └── login/
│   │   └── lessons/
│   │       ├── lesson-list/
│   │       └── lesson-detail/
│   ├── app.routes.ts
│   └── app.component.ts
├── styles/                     <- Design System (Sass/SCSS)
│   ├── 1-settings/             <- Variables, tokens
│   │   ├── _colors.scss
│   │   ├── _typography.scss
│   │   ├── _spacing.scss
│   │   └── _breakpoints.scss
│   ├── 2-atoms/
│   ├── 3-molecules/
│   ├── 4-organisms/
│   ├── 5-templates/
│   └── main.scss               <- Punto de entrada, importa todo
└── assets/
```

4.2 Backend (NestJS) — Arquitectura Modular

```
backend/src/
├── auth/                       <- Modulo de autenticacion
│   ├── dto/
│   │   └── auth.dto.ts
│   ├── guards/
```

```
│   │   └── jwt-auth.guard.ts    <- Protege rutas privadas
│   ├── strategies/
│   │   ├── jwt.strategy.ts
│   │   └── google.strategy.ts
│   ├── auth.controller.ts
│   ├── auth.service.ts
│   └── auth.module.ts
├── lessons/                     <- Modulo de lecciones
│   ├── dto/
│   │   └── create-lesson.dto.ts
│   ├── schemas/
│   │   └── lesson.schema.ts     <- Schema de Mongoose
│   ├── lessons.controller.ts
│   ├── lessons.service.ts
│   └── lessons.module.ts
├── app.module.ts                <- Modulo raiz
└── main.ts
```

5. Conectar el Repositorio Git

Tienes el repositorio ya creado. Sigue estos pasos exactos para conectar el proyecto local al repo remoto siguiendo GitFlow simplificado (tal como exige el PPT).

5.1 Inicializar Git y conectar al repositorio

```
# Desde la carpeta raiz codex-fct-2026/

# Inicializa Git
git init

# Conecta con tu repositorio (sustituye la URL por la tuya)
git remote add origin https://github.com/tu-usuario/codex-fct-
2026.git

# Verifica que se ha conectado correctamente
git remote -v
```

5.2 Primer commit y push

```
# Añade todos los archivos al staging
git add .

# Crea el primer commit
git commit -m "feat: scaffolding inicial Angular + NestJS"

# Sube a main (rama principal)
git push -u origin main
```

5.3 Crear las ramas de GitFlow

Según el PPT, el flujo de Git debe seguir esta estructura:

```
# Crea la rama develop (integracion)
git checkout -b develop
git push -u origin develop

# Para cada nueva funcionalidad, crea una rama feature:
git checkout -b feature/nombre-de-la-tarea

# Ejemplo real:
git checkout -b feature/auth-google-login
git checkout -b feature/lesson-list-component
git checkout -b feature/markdown-renderer
```

Regla del PPT: Pull Requests obligatorias

Cada subida a develop o main DEBE hacerse mediante una Pull Request revisada por un compañero. NUNCA hagas push directo a main. El flujo es: feature/* -> Pull Request -> develop -> Pull Request -> main

5.4 Archivo .gitignore

Crea un archivo .gitignore en la raíz del proyecto con el siguiente contenido para evitar subir archivos innecesarios:

```
# Node
node_modules/

# Angular build
frontend/dist/
frontend/.angular/

# NestJS build
backend/dist/

# Variables de entorno (NUNCA subir al repo)
.env
*.env.local

# IDE
.vscode/
.idea/

# OS
.DS_Store
Thumbs.db
```

6. Primeros Pasos tras el Scaffolding

Una vez creado y conectado el proyecto, estos son los comandos del día a día que usarás durante el desarrollo:

6.1 Arrancar los servidores de desarrollo

```
# Terminal 1 -> Frontend Angular (http://localhost:4200)
cd frontend
ng serve

# Terminal 2 -> Backend NestJS (http://localhost:3000)
cd backend
npm run start:dev
```

6.2 Comandos Angular útiles para el proyecto

```
# Generar un componente standalone (Atomic Design)
ng generate component shared/atoms/button --standalone

# Generar un servicio
ng generate service core/services/auth

# Generar un guard funcional (requerido por PPT)
ng generate guard core/guards/auth --functional

# Generar una pagina (feature)
ng generate component features/lessons/lesson-list --standalone
```

6.3 Comandos NestJS útiles para el proyecto

```
# Generar un modulo completo (controller + service + module)
nest generate module lessons
nest generate controller lessons
nest generate service lessons

# O todo de una vez:
nest generate resource lessons  # Genera CRUD completo

# Generar modulo de auth:
nest generate resource auth
```

6.4 Verificar el build de producción

```
# Frontend - build optimizado
cd frontend && ng build

# Backend - compilar TypeScript
cd backend && npm run build
```

7. Checklist Final de Verificación

Antes de empezar a desarrollar funcionalidades, asegúrate de que todo está correcto marcando estos puntos:

✓	Tarea de verificación
☐	node --version y npm --version muestran versiones correctas
☐	ng version muestra Angular CLI 21.x
☐	nest --version muestra NestJS CLI
☐	La carpeta codex-fct-2026/ existe con frontend/ y backend/ dentro
☐	ng serve arranca el frontend en http://localhost:4200 sin errores
☐	npm run start:dev arranca el backend en http://localhost:3000 sin errores
☐	git remote -v muestra la URL de tu repositorio
☐	Las ramas main y develop existen en el repositorio remoto
☐	El archivo .gitignore está creado en la raíz
☐	La estructura de carpetas de styles/ (Atomic Design) está creada en frontend/src/

🚀 **Próximos pasos recomendados**

1. Configurar el Design System: crear las variables SCSS en styles/1-settings/. 2. Implementar CU-01 (Login con Google). 3. Crear el primer componente Atom: el Botón. 4. Conectar MongoDB Atlas en el backend con la cadena de conexión.